

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ESCOLA POLITÉCNICA
BACHARELADO EM ENGENHARIA DE SOFTWARE

Caio Henrique
Gabriel Moribe
Guilherme Celente
Leonardo Berlanda

PAYFLOW: DOCUMENTAÇÃO DE ARQUITETURA DE UMA API DE
PAGAMENTOS EM NUVEM

Documentação parcial conforme arc42, C4 Model, UML, DER e Software
Architecture Canvas

CURITIBA
2026

Caio Henrique
Gabriel Moribe
Guilherme Celente
Leonardo Berlanda

**PAYFLOW: DOCUMENTAÇÃO DE ARQUITETURA DE UMA API DE
PAGAMENTOS EM NUVEM**

Trabalho parcial apresentado à disciplina de Arquitetura de Soluções com Cloud, do curso de Bacharelado em Engenharia de Software da Pontifícia Universidade Católica do Paraná, como requisito parcial de avaliação.

Professor: Manoel Valerio da Silveira Neto

CURITIBA
2026

SUMÁRIO

1 INTRODUÇÃO E METAS	4
1.1 Visão geral dos requisitos	4
1.2 Metas de qualidade	4
1.3 Partes interessadas	5
1.4 Software Architecture Canvas	5
2 RESTRIÇÕES DE ARQUITETURA	6
3 CONTEXTO E ESCOPO	7
3.1 Contexto de negócio	7
3.2 Contexto técnico	9
4 ESTRATÉGIA DE SOLUÇÃO	9
4.1 Decomposição de nível superior: diagrama de contêineres	9
5 VISÃO DE BLOCOS DE CONSTRUÇÃO	11
5.1 Nível 1: caixa branca do sistema	11
5.2 Nível 2: componentes da API de Pagamentos	11
5.3 Diagrama de componentes UML	13
5.4 Nível 3: código (C4 Model, nível 4) – diagrama de classes	14
6 VISÃO DE EXECUÇÃO	15
6.1 Cenário 1: pagamento com cartão e notificação	15
6.2 Cenário 2: pagamento via PIX	16
6.3 Cenário 3: falha do gateway externo	17
6.4 Cenário 4: reprocessamento da DLQ	17
6.5 Escalonamento com KEDA	17
7 VISÃO DE IMPLANTAÇÃO	18
8 CONCEITOS TRANSVERSAIS	19
8.1 Modelo de dados	19
8.2 Segurança	20
8.3 Idempotência	21
8.4 Processamento assíncrono e tolerância a falhas	21
8.5 Observabilidade	21
9 DECISÕES DE ARQUITETURA	21
10 REQUISITOS DE QUALIDADE	22
10.1 Árvore de qualidade	22

10.2 Cenários de qualidade	23
11 RISCOS E DÍVIDAS TÉCNICAS	24
12 GLOSSÁRIO.....	25

1 INTRODUÇÃO E METAS

O PayFlow é uma API de pagamentos que intermedeia a relação entre lojas virtuais e um gateway de pagamento externo. A loja envia um pedido de pagamento; o PayFlow valida a requisição, garante que ela não seja processada duas vezes, autoriza a operação no gateway externo, registra o resultado e notifica a loja de forma assíncrona. O objetivo é oferecer ao lojista uma integração única, confiável e que escale automaticamente com a demanda.

1.1 Visão geral dos requisitos

Quadro 1 – Requisitos funcionais

ID	Requisito	Descrição
RF01	Autenticação do lojista	Autenticar cada requisição por API Key associada a um lojista cadastrado.
RF02	Pagamento com cartão	Criar um pagamento com cartão tokenizado pelo gateway externo e autorizá-lo de forma síncrona.
RF03	Pagamento via PIX	Criar uma cobrança PIX no gateway externo, devolver o QR Code e confirmar o pagamento quando o gateway notificar.
RF04	Consulta de pagamento	Consultar o status de um pagamento por identificador.
RF05	Estorno	Estornar integralmente um pagamento autorizado ou confirmado.
RF06	Idempotência	Tratar requisições repetidas com a mesma Idempotency-Key devolvendo a resposta original, sem cobrar duas vezes.
RF07	Notificação por webhook	Notificar a loja sobre cada mudança de status por webhook assinado (HMAC), com retentativas.
RF08	Tratamento de falhas de notificação	Encaminhar para uma dead letter queue as notificações que falharem após cinco tentativas, permitindo reproprocessamento.
RF09	Recepção de confirmações do gateway	Receber e validar webhooks do gateway externo que confirmam pagamentos assíncronos.

Fonte: os autores (2026).

1.2 Metas de qualidade

Quadro 2 – Metas de qualidade prioritárias

Prioridade	Meta	Cenário resumido
1	Segurança	Nenhum dado de cartão trafega ou é armazenado na API: a tokenização é feita pelo gateway externo. Comunicação sempre em TLS e webhooks assinados.

Prioridade	Meta	Cenário resumido
2	Confiabilidade	Nenhuma cobrança duplicada e nenhuma notificação perdida, com idempotência no Redis e DLQ na SQS.
3	Escalabilidade	A API e o worker escalam automaticamente pela KEDA conforme carga e tamanho da fila; o worker escala a zero quando ocioso.
4	Desempenho	Criação de pagamento com cartão em até 2 s (p95); webhook entregue em até 30 s.
5	Manutenibilidade	O gateway externo é um adaptador substituível sem alterar o núcleo da API.

Fonte: os autores (2026).

1.3 Partes interessadas

Quadro 3 – Partes interessadas e expectativas

Papel	Descrição	Expectativas
Lojista	Comerciante que integra a API	Integração simples, resposta rápida e notificações confiáveis.
Comprador	Cliente final da loja	Pagamento rápido e seguro.
Equipe de desenvolvimento	Quatro integrantes do grupo	Arquitetura simples de operar e de evoluir.
Gateway de pagamento externo	Provedor de autorização (cartão e PIX)	Uso correto da API, dos webhooks e das credenciais.
Professor / avaliador	Disciplina de Arquitetura de Soluções com Cloud	Documentação clara nos padrões arc42, C4 e UML.

Fonte: os autores (2026).

1.4 Software Architecture Canvas

O Quadro 4 apresenta o Software Architecture Canvas (ARC42, 2024): objetivos em verde, solução em amarelo e avaliação em azul. Cada bloco indica a seção do arc42 correspondente.

Quadro 4 – Software Architecture Canvas do PayFlow

VISÃO GERAL FUNCIONAL (1.1)	METAS DE QUALIDADE (1.2)	CONTEXTO DE NEGÓCIO (3.1)
- Pagamento com cartão e PIX - Consulta e estorno - Idempotência - Webhooks com tentativas e DLQ	- Segurança – sem dados de cartão na API - Confiabilidade – sem duplicidade nem perda - Escalabilidade automática (KEDA) - Desempenho p95 ≤ 2 s	- Lojas virtuais dos lojistas - Compradores - Gateway de pagamento externo

RESTRIÇÕES ORGANIZACIONAIS (2) • Equipe de 4 pessoas, um semestre • Sprints de 2 semanas • Créditos educacionais AWS • Documentação arc42 e C4	RESTRIÇÕES TÉCNICAS (2) • AWS sa-east-1 • Kubernetes (EKS) e KEDA • Java 21 / Spring Boot • Redis, SQS e ALB gerenciados	ESTRATÉGIA DE SOLUÇÃO (4, 9) • API síncrona + worker assíncrono • Autorização delegada ao gateway externo • Idempotência no Redis • SQS com DLQ para notificações
DESAFIOS E RISCOS (11) • Dependência do gateway externo • Redis como único armazenamento • Entrega de webhooks a lojas instáveis		MEDIDAS DE AVALIAÇÃO (10) • Cenários CQ-01 a CQ-11 • Teste de carga com k6 • Métricas de fila e de latência

Fonte: os autores (2026), adaptado de arc42 (2024).

2 RESTRIÇÕES DE ARQUITETURA

Quadro 5 – Restrições técnicas

Restrição	Explicação
Nuvem AWS, região sa-east-1	Menor latência com o gateway externo e as lojas brasileiras; uso de créditos educacionais.
Kubernetes (Amazon EKS) com KEDA	Toda a aplicação executa em contêineres; o autoescalonamento é feito pela KEDA, não por HPA manual.
Serviços gerenciados	Load balancer (ALB), Redis (ElastiCache) e filas (SQS) são gerenciados pela AWS, sem administração de servidores.
Sem dados de cartão na API	A tokenização do cartão é feita pelo gateway externo; o PayFlow recebe apenas o token, mantendo-se fora do escopo PCI DSS de armazenamento.
Java 21 e Spring Boot	Padrão da equipe para a API e o worker.
TLS 1.2 ou superior	Em todas as comunicações externas.

Fonte: os autores (2026).

Quadro 6 – Restrições organizacionais e convenções

Restrição	Explicação
Equipe e prazo	Quatro estudantes durante o semestre 2/2026, com sprints de duas semanas.
Escopo parcial	Nesta entrega, os princípios de arquitetura em nuvem ainda não são avaliados em sua totalidade.
Documentação	arc42 em português; código e identificadores em inglês; diagramas em PlantUML versionados.
API	REST/JSON, versionada em /v1, erros no formato Problem Details (RFC 9457).
Decisões	Registradas como ADRs (seção 9).

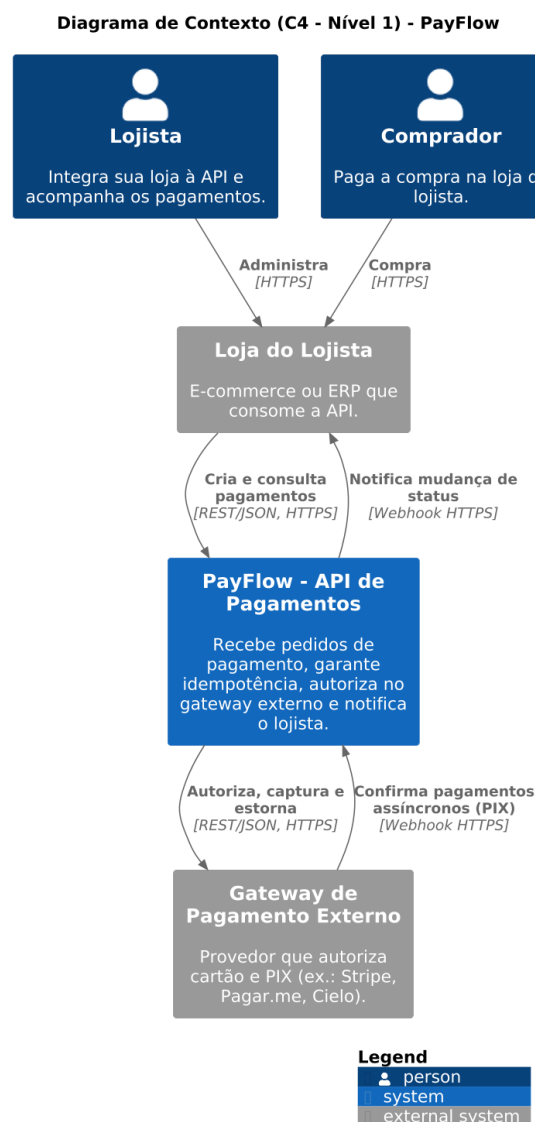
Fonte: os autores (2026).

3 CONTEXTO E ESCOPO

3.1 Contexto de negócio

A Figura 1 mostra o diagrama de contexto (C4 Model, nível 1). O PayFlow é usado pela loja do lojista e se comunica com um único sistema externo: o gateway de pagamento, que autoriza cartão e PIX. O comprador não interage diretamente com o PayFlow.

Figura 1 – Diagrama de contexto (C4 Model, nível 1)



Fonte: os autores (2026).

Quadro 7 – Parceiros de comunicação

Parceiro	Entradas para o PayFlow	Saídas do PayFlow
Loja do Lojista	Criação, consulta e estorno de pagamentos	Resposta síncrona com status; webhooks de mudança de status
Gateway de pagamento externo	Resultado da autorização; confirmação assíncrona de PIX	Pedidos de autorização, cobrança PIX e estorno

Fonte: os autores (2026).

3.2 Contexto técnico

Quadro 8 – Canais e protocolos

Interface	Canal	Protocolo e formato	Segurança
Loja → PayFlow	Internet, ALB	HTTPS, REST/JSON (OpenAPI 3.1)	API Key por lojista; Idempotency-Key
PayFlow → Loja (webhook)	Internet	HTTPS POST, JSON	Assinatura HMAC-SHA256, timestamp anti-replay
PayFlow → Gateway externo	Internet	HTTPS, REST/JSON	Credenciais do gateway (Secrets Manager), TLS
Gateway externo → PayFlow	Internet, ALB	HTTPS POST, JSON	Assinatura verificada pela API

Fonte: os autores (2026).

4 ESTRATÉGIA DE SOLUÇÃO

Quadro 9 – Metas de qualidade e abordagens

Meta	Abordagem	Referência
Segurança	Tokenização delegada ao gateway externo; API Key por lojista; webhooks assinados; segredos no AWS Secrets Manager.	ADR-02
Confiabilidade	Idempotência com SETNX no Redis; publicação de evento após a persistência; SQS com tentativas e DLQ.	ADR-03, ADR-04
Escalabilidade	API e worker em Kubernetes escalados pela KEDA: a API por req/s e CPU, o worker pelo tamanho da fila (até zero réplicas).	ADR-01, ADR-05
Desempenho	Caminho síncrono curto (Redis + gateway); notificações fora do caminho crítico, via fila.	ADR-04
Manutenibilidade	Arquitetura hexagonal: gateway externo, Redis e SQS são adaptadores de portas do domínio.	ADR-02

Fonte: os autores (2026).

Contêiner	Tecnologia	Responsabilidade
Load Balancer	AWS Application Load Balancer	Terminação TLS, health checks e distribuição das requisições.
Cluster Kubernetes	Amazon EKS	Executa os pods da API, do worker e da KEDA.
API de Pagamentos	Java 21, Spring Boot	Valida, garante idempotência, autoriza no gateway, persiste e publica eventos.

Contêiner	Tecnologia	Responsabilidade
Worker de Notificações	Java 21, Spring Boot	Consome a fila e entrega webhooks ao lojista com retentativas.
KEDA	Kubernetes Event-driven Autoscaling	Escala a API (req/s, CPU) e o worker (tamanho da fila SQS).
Redis	Amazon ElastiCache	Pagamentos, chaves de idempotência e respostas armazenadas.
Fila de Eventos	Amazon SQS (standard)	Eventos de pagamento para notificação assíncrona.
Dead Letter Queue	Amazon SQS	Mensagens não processadas após cinco tentativas.
Gateway de pagamento externo	Sistema externo	Tokenização, autorização de cartão, cobrança PIX e estorno.

Fonte: os autores (2026).

5 VISÃO DE BLOCOS DE CONSTRUÇÃO

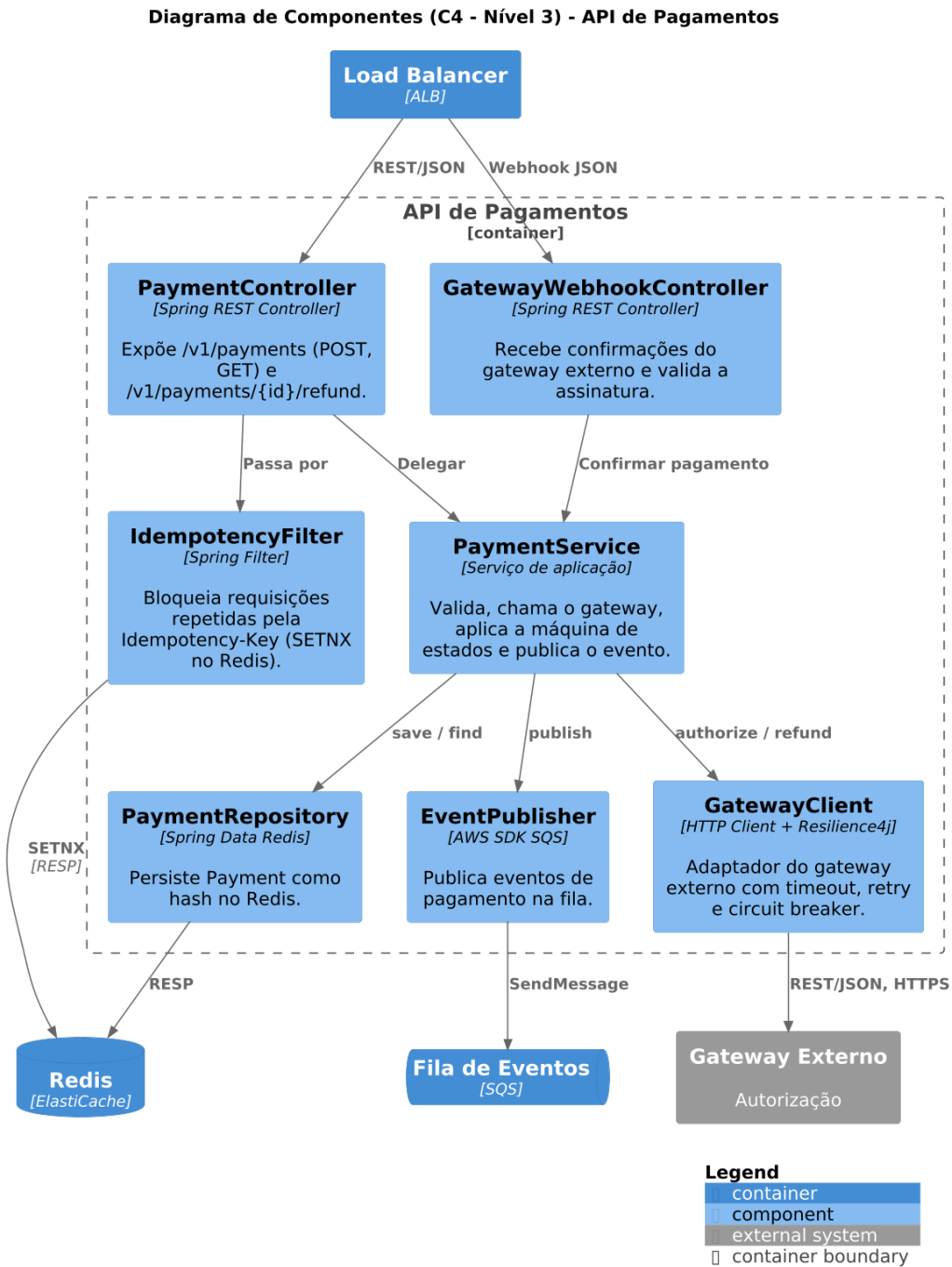
5.1 Nível 1: caixa branca do sistema

A caixa branca do PayFlow corresponde aos contêineres da Figura 2 e do Quadro 10. A decomposição separa o caminho síncrono (API) do assíncrono (worker), isola o gateway externo atrás de um adaptador e usa serviços gerenciados para balanceamento, persistência e filas.

5.2 Nível 2: componentes da API de Pagamentos

A Figura 3 detalha a API de Pagamentos (C4 Model, nível 3). Os controladores e o filtro de idempotência formam a entrada; o PaymentService concentra a regra de negócio; GatewayClient, PaymentRepository e EventPublisher são os adaptadores de saída.

Figura 3 – Diagrama de componentes da API de Pagamentos (C4 Model, nível 3)



Fonte: os autores (2026).

Quadro 11 – Componentes da API de Pagamentos

Componente	Responsabilidade
PaymentController	Endpoints POST /v1/payments, GET /v1/payments/{id} e POST /v1/payments/{id}/refund.
GatewayWebhookController	Recebe confirmações do gateway externo e valida a assinatura.
IdempotencyFilter	Registra a Idempotency-Key no Redis (SETNX, TTL 24 h) e devolve a resposta original em repetições.

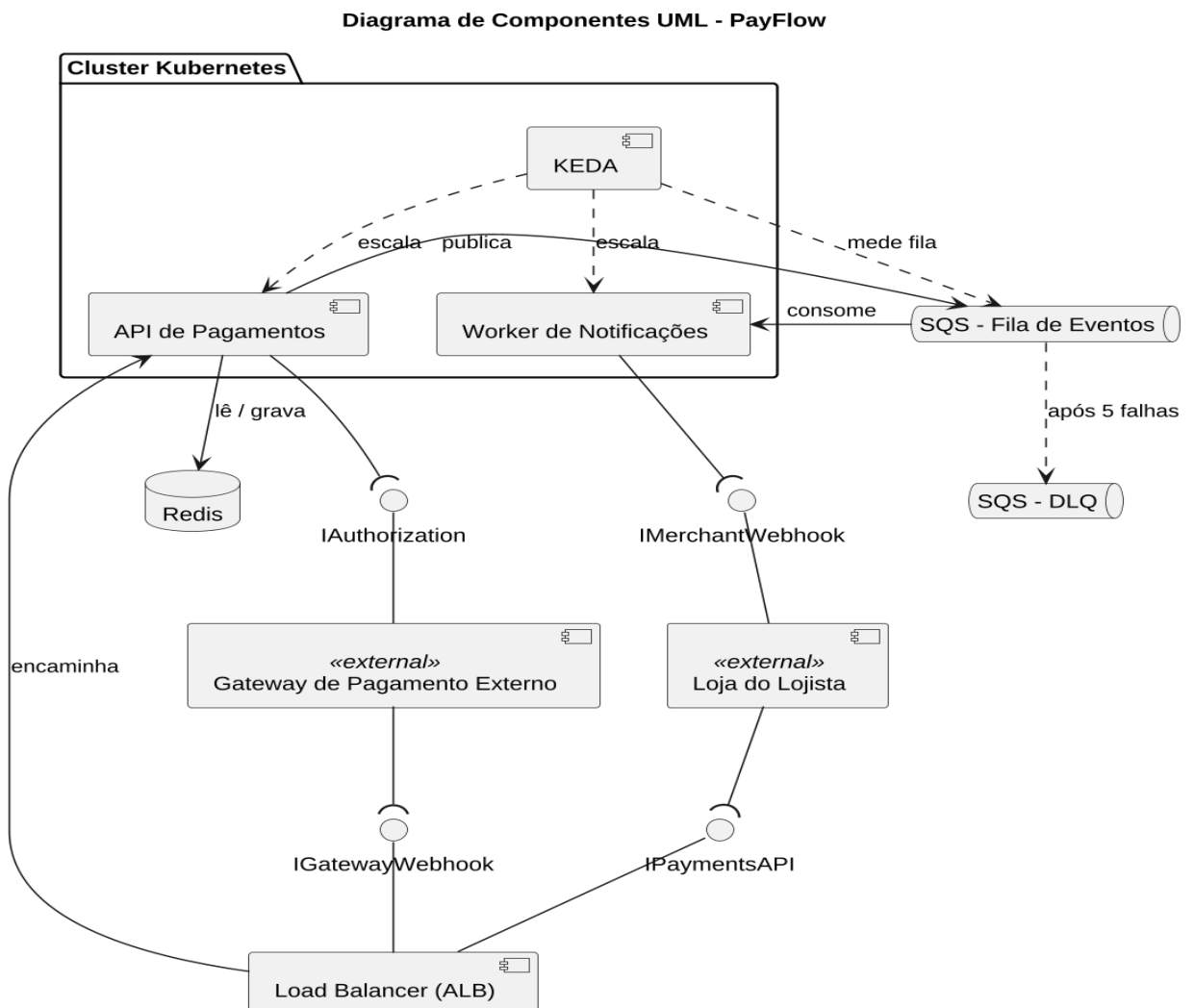
Componente	Responsabilidade
PaymentService	Valida o pedido, chama o gateway, aplica a máquina de estados e publica o evento.
GatewayClient	Adaptador HTTP do gateway externo com timeout de 5 s, retentativa e circuit breaker (Resilience4j).
PaymentRepository	Persiste o pagamento como hash no Redis (Spring Data Redis).
EventPublisher	Publica eventos na fila SQS (AWS SDK).

Fonte: os autores (2026).

5.3 Diagrama de componentes UML

A Figura 4 apresenta os componentes do sistema com suas interfaces providas e requeridas: IPaymentsAPI e IGatewayWebhook (expostas pelo load balancer), IAuthorization (provida pelo gateway externo) e IMerchantWebhook (provida pela loja).

Figura 4 – Diagrama de componentes UML

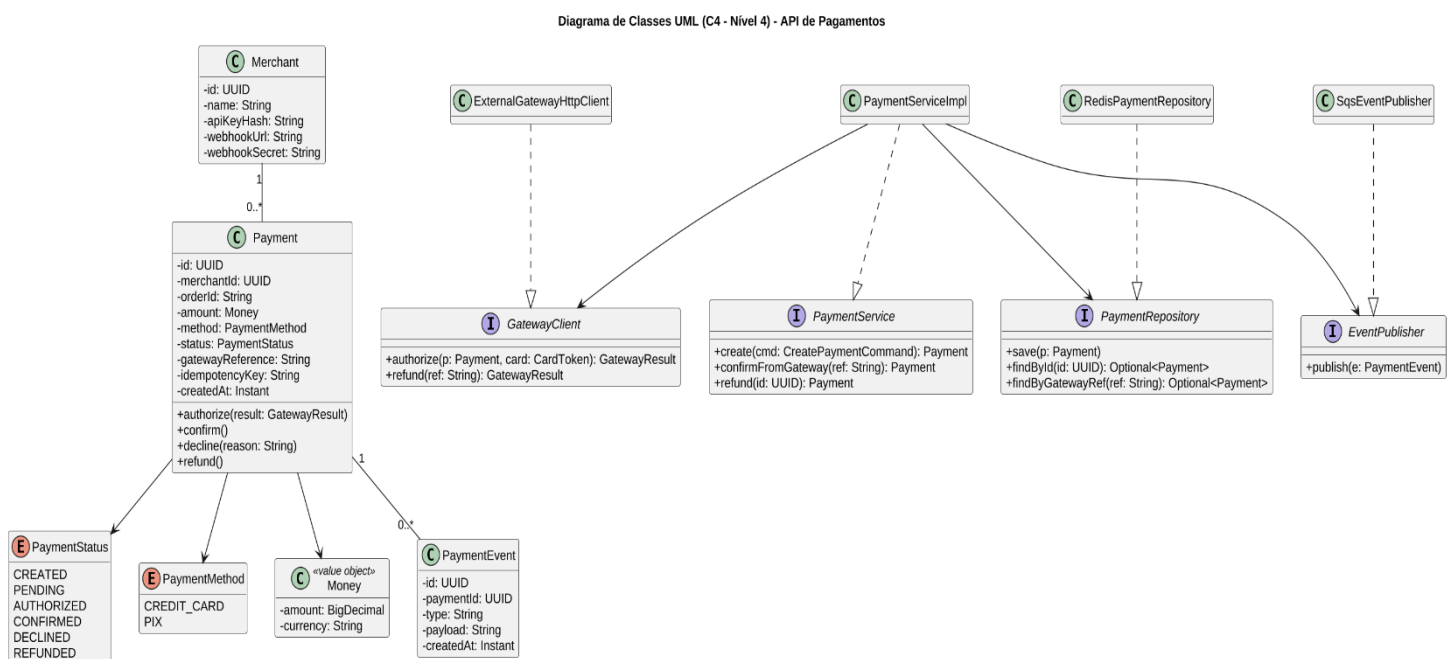


Fonte: os autores (2026).

5.4 Nível 3: código (C4 Model, nível 4) – diagrama de classes

A Figura 5 mostra as classes da API. Payment é o agregado principal, com Money como objeto de valor e PaymentEvent como registro de notificação. As interfaces GatewayClient, PaymentRepository e EventPublisher são as portas do domínio, implementadas pelos adaptadores ExternalGatewayHttpClient, RedisPaymentRepository e SqsEventPublisher.

Figura 5 – Diagrama de classes UML (C4 Model, nível 4)



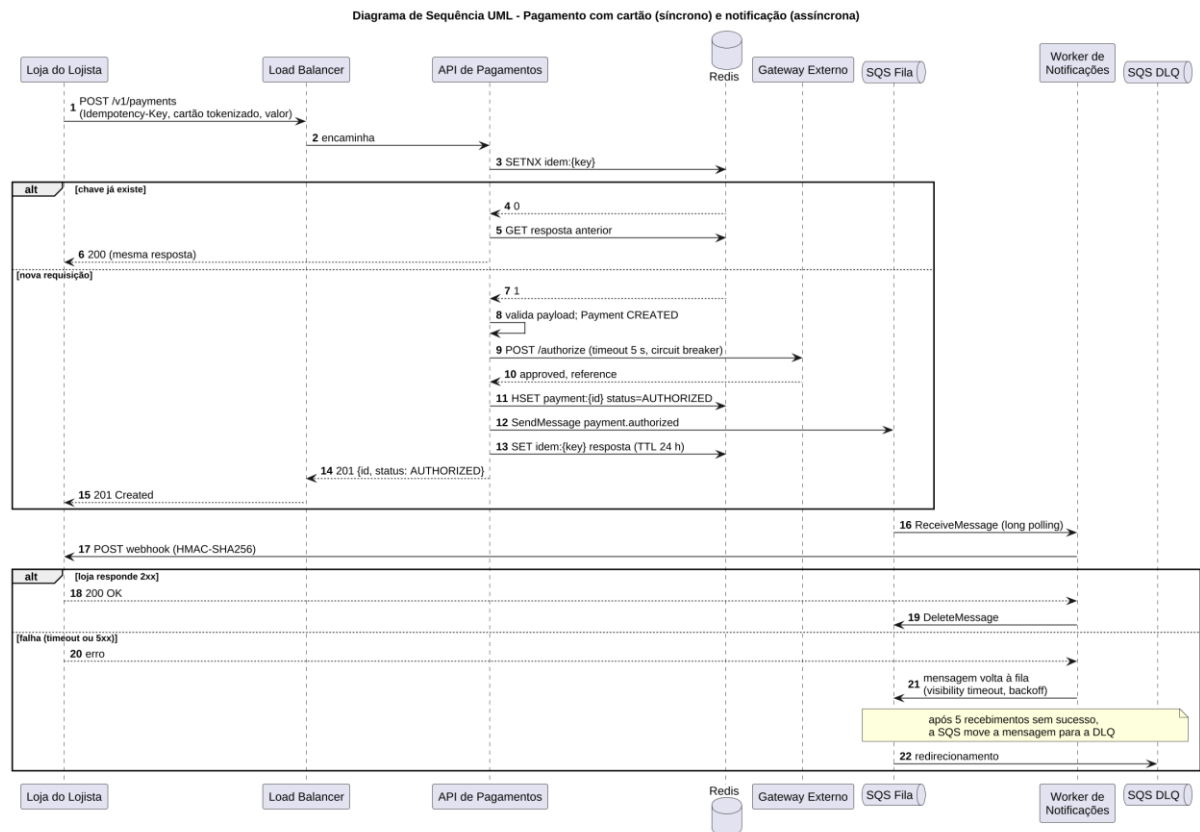
Fonte: os autores (2026).

6 VISÃO DE EXECUÇÃO

6.1 Cenário 1: pagamento com cartão e notificação

A Figura 6 mostra o fluxo completo. A parte síncrona vai da requisição da loja até a resposta 201: verificação de idempotência, autorização no gateway externo, gravação no Redis e publicação do evento. A parte assíncrona é executada pelo worker: entrega do webhook, retorno da mensagem à fila em caso de falha e redirecionamento para a DLQ após cinco tentativas.

Figura 6 – Diagrama de sequência UML – pagamento com cartão e notificação

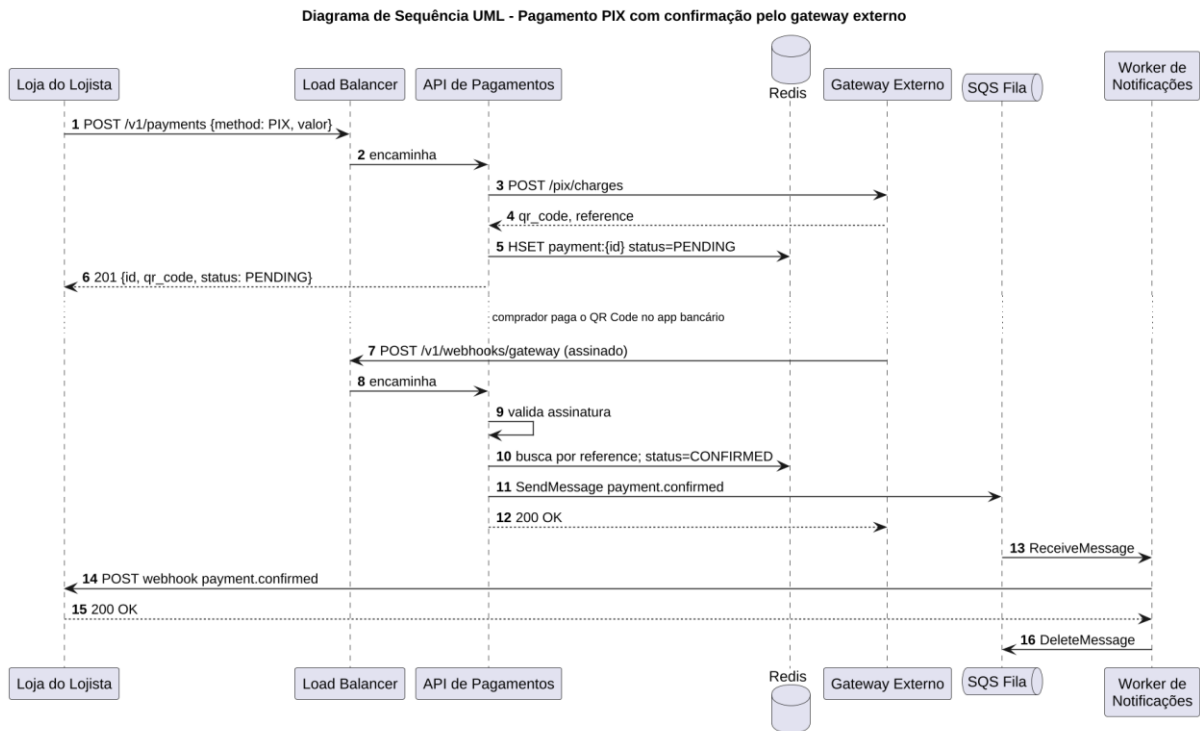


Fonte: os autores (2026).

6.2 Cenário 2: pagamento via PIX

A Figura 7 mostra o PIX, cuja confirmação é assíncrona: a API cria a cobrança no gateway, devolve o QR Code com status PENDING e, ao receber o webhook do gateway, muda o status para CONFIRMED e publica o evento para notificar a loja.

Figura 7 – Diagrama de sequência UML – pagamento via PIX



Fonte: os autores (2026).

6.3 Cenário 3: falha do gateway externo

Se o gateway externo não responder em 5 s ou retornar erro 5xx, o GatewayClient faz uma tentativa. Persistindo a falha, o circuit breaker abre por 30 s e a API responde 503 com cabeçalho Retry-After; a loja pode repetir a requisição com a mesma Idempotency-Key sem risco de cobrança duplicada.

6.4 Cenário 4: reproprocessamento da DLQ

Mensagens na DLQ ficam retidas por 14 dias. Um operador pode acionar o redrive da SQS para devolvê-las à fila principal quando a loja voltar a responder. Um alarme no CloudWatch dispara quando a DLQ tem mais de dez mensagens.

6.5 Escalonamento com KEDA

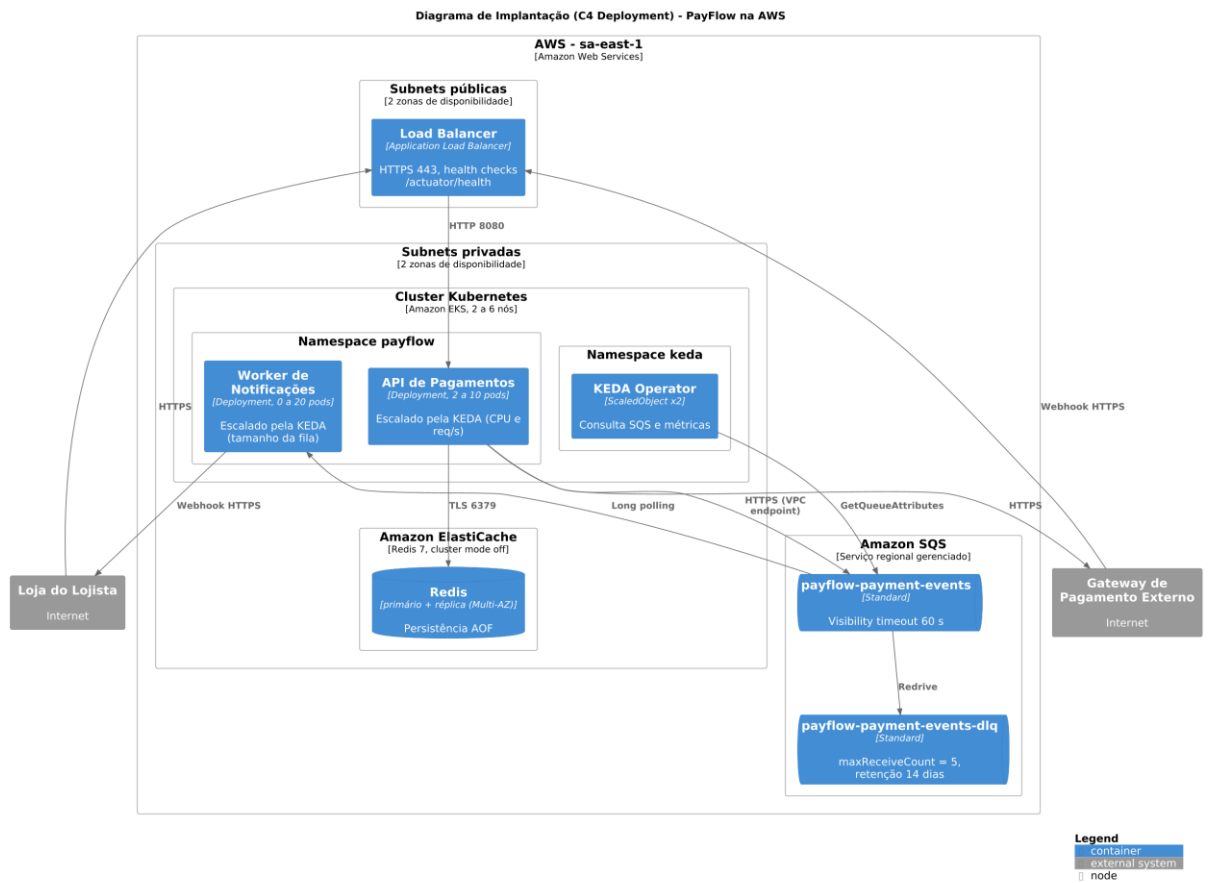
A KEDA mantém dois ScaledObjects. O da API escala de 2 a 10 pods quando a taxa de requisições por pod ultrapassa 50 req/s ou a CPU passa de 70%. O do worker escala de 0 a 20 pods proporcionalmente ao número de mensagens visíveis

na fila (uma réplica a cada 100 mensagens) e volta a zero após cinco minutos sem mensagens.

7 VISÃO DE IMPLANTAÇÃO

A Figura 8 apresenta a implantação na AWS. O load balancer fica nas subnets públicas; o cluster EKS e o Redis, nas subnets privadas; a SQS é um serviço regional acessado por VPC endpoint. Os pods da API e do worker executam no namespace payflow e a KEDA no namespace keda.

Figura 8 – Diagrama de implantação na AWS (C4 Model – Deployment)



Fonte: os autores (2026).

Quadro 12 – Elementos de infraestrutura

Elemento	Serviço AWS	Configuração
Load balancer	Application Load Balancer	HTTPS 443, certificado ACM, health check em /actuator/health.
Cluster Kubernetes	Amazon EKS	2 a 6 nós em duas zonas de disponibilidade; Cluster Autoscaler.

Elemento	Serviço AWS	Configuração
Autoescalonamento	KEDA (instalada via Helm)	Scaler de SQS para o worker; scaler de CPU e Prometheus (req/s) para a API.
Redis	Amazon ElastiCache	Redis 7, primário e réplica em zonas distintas, persistência AOF, TLS.
Fila e DLQ	Amazon SQS	Fila standard com visibility timeout 60 s; DLQ com maxReceiveCount 5 e retenção de 14 dias.
Segredos	AWS Secrets Manager	Credenciais do gateway externo e segredos de webhook, injetados nos pods.
Observabilidade	CloudWatch, Prometheus	Logs, métricas de latência, tamanho da fila e alarme de DLQ.
Entrega contínua	GitHub Actions, Amazon ECR, Terraform	Build, testes, publicação de imagens e infraestrutura como código.

Fonte: os autores (2026).

Quadro 13 – Ambientes

Ambiente	Características
Desenvolvimento	Docker Compose com Redis e LocalStack (SQS); gateway externo em modo sandbox.
Homologação	Cluster EKS reduzido (1 nó), gateway externo em sandbox, dados sintéticos.
Produção	Duas zonas de disponibilidade, Redis com réplica, alarmes e backups.

Fonte: os autores (2026).

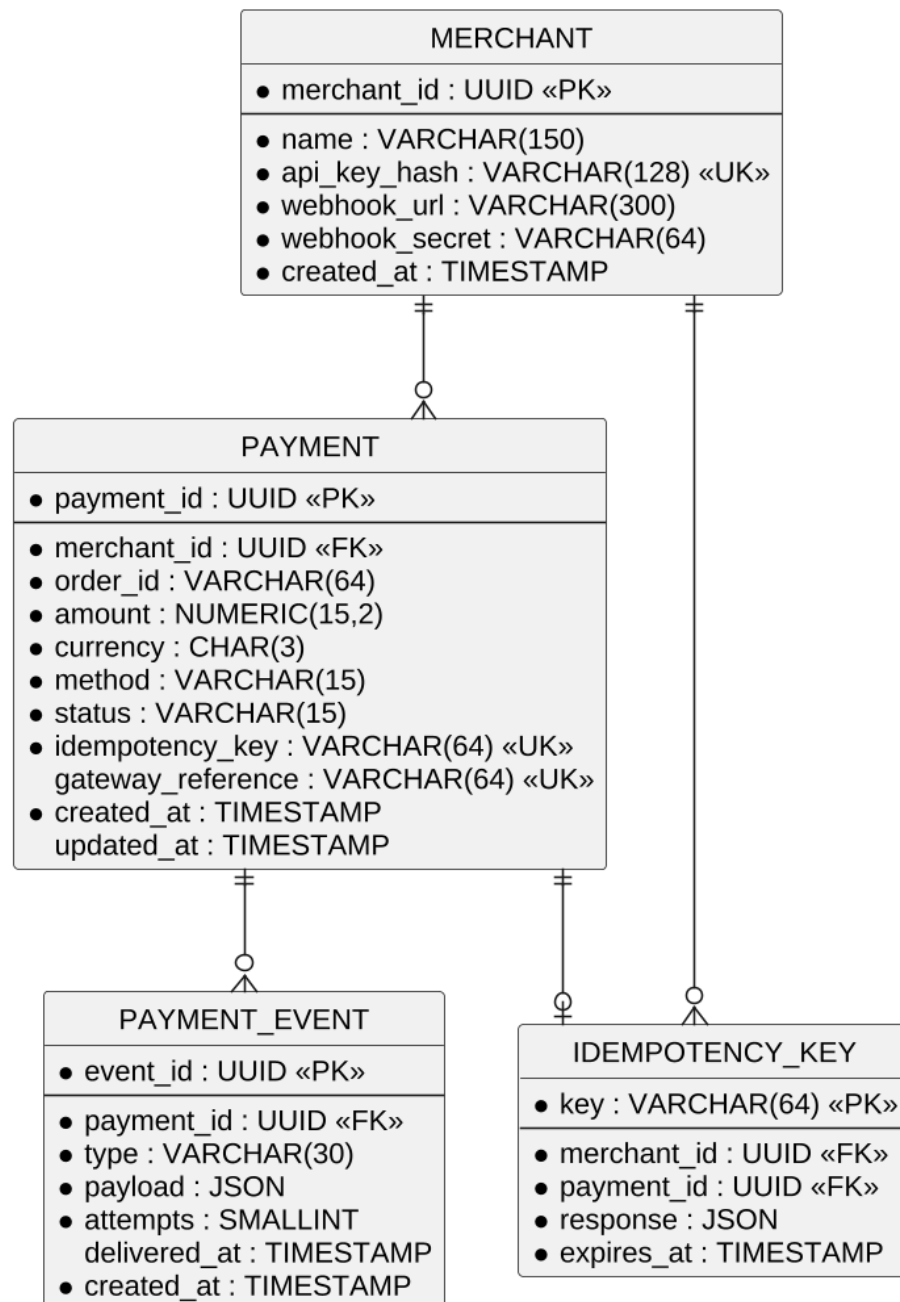
8 CONCEITOS TRANSVERSAIS

8.1 Modelo de dados

A Figura 9 apresenta o DER do modelo lógico. Os dados são armazenados no Redis como hashes (payment:{id}, merchant:{id}), conjuntos por lojista e chaves de idempotência com TTL. O modelo é relacional na lógica para permitir migração futura a um banco relacional (ver DT-01).

Figura 9 – Diagrama de Entidades e Relacionamentos (modelo lógico)

Diagrama de Entidades e Relacionamentos (DER) - Modelo lógico do PayFlow



Fonte: os autores (2026).

8.2 Segurança

A API nunca recebe o número do cartão: a loja o tokeniza diretamente no gateway externo e envia apenas o token. Cada lojista se autentica por API Key (armazenada como hash). Webhooks enviados e recebidos são assinados com

HMAC-SHA256 e carregam timestamp para evitar replay. Segredos ficam no Secrets Manager e todas as conexões usam TLS.

8.3 Idempotência

Toda criação de pagamento e estorno exige o cabeçalho Idempotency-Key. O filtro executa SETNX no Redis; se a chave já existir, devolve a resposta armazenada. Chaves expiram em 24 horas.

8.4 Processamento assíncrono e tolerância a falhas

Eventos são publicados na SQS somente após a gravação do pagamento no Redis. O worker é idempotente (usa o identificador do evento) e só apaga a mensagem após resposta 2xx da loja. Falhas devolvem a mensagem à fila; após cinco recebimentos a SQS a move para a DLQ. Chamadas ao gateway externo usam timeout, retentativa e circuit breaker.

8.5 Observabilidade

Logs estruturados em JSON com identificador de correlação, métricas de latência por endpoint, tamanho da fila e da DLQ, e alarmes no CloudWatch.

9 DECISÕES DE ARQUITETURA

O Quadro 14 registra as decisões no formato ADR (NYGARD, 2011).

Quadro 14 – Registros de decisão de arquitetura (ADRs)

ID	Título	Contexto	Decisão	Estado	Consequências
ADR-01	Kubernetes (EKS) com KEDA para autoescalonamento	A carga varia ao longo do dia e as notificações chegam em rajadas.	Executar API e worker no EKS e escalar com KEDA: a API por req/s e CPU; o worker pelo tamanho da fila SQS, inclusive até zero réplicas.	Aceita	Escala automática e custo proporcional ao uso; exige operar o cluster e a KEDA.

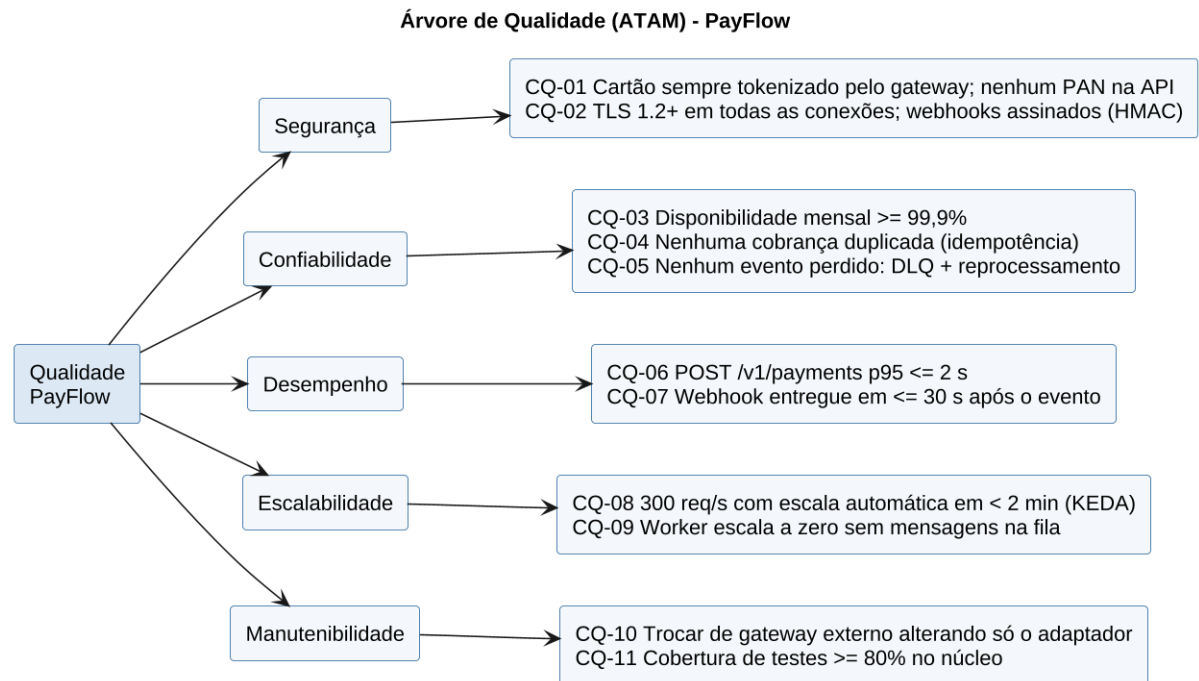
ID	Título	Contexto	Decisão	Estado	Consequências
ADR-02	Delegar tokenização e autorização a um gateway externo	Certificação PCI DSS e integração direta com adquirentes são inviáveis para a equipe e o prazo.	Integrar um único gateway externo por meio da porta GatewayClient, mantendo a API fora do escopo de armazenamento de cartão.	Aceita	Redução drástica de escopo e risco; dependência de um fornecedor e de suas taxas.
ADR-03	Redis como armazenamento e mecanismo de idempotência	Simplicidade operacional e baixa latência para uma entrega parcial.	Usar ElastiCache Redis com persistência AOF e réplica para pagamentos e chaves de idempotência (SETNX + TTL).	Aceita	Baixa latência e operação simples; consultas limitadas e risco de perda em falha extrema (DT-01).
ADR-04	Amazon SQS com DLQ para notificações	Webhooks não podem atrasar a resposta da API nem se perder quando a loja falha.	Publicar eventos em fila SQS standard consumida pelo worker; DLQ com maxReceiveCount 5 e redrive manual.	Aceita	Entrega desacoplada e recuperável; ordem não garantida (aceitável, pois eventos carregam timestamp).
ADR-05	Application Load Balancer gerenciado	Necessidade de TLS, health checks e distribuição de carga sem operar proxies.	Usar ALB como ponto de entrada único, integrado ao EKS pelo AWS Load Balancer Controller.	Aceita	Menor esforço operacional; custo por hora e por requisição.

Fonte: os autores (2026).

10 REQUISITOS DE QUALIDADE

10.1 Árvore de qualidade

A Figura 10 organiza os atributos de qualidade e seus cenários (CQ-01 a CQ-11), conforme o método ATAM (BASS; CLEMENTS; KAZMAN, 2021).

Figura 10 – Árvore de qualidade (ATAM)

10.2 Cenários de qualidade

ID	Atributo	Cenário	Medida
CQ-01	Segurança	A loja envia um pagamento; a API recebe apenas o token gerado pelo gateway externo.	0 ocorrências de PAN em logs ou no Redis
CQ-02	Segurança	Uma loja recebe um webhook e valida a assinatura.	100% dos webhooks assinados; TLS 1.2+
CQ-03	Confiabilidade	Uma zona de disponibilidade falha; a API continua respondendo.	Disponibilidade mensal $\geq 99,9\%$
CQ-04	Confiabilidade	A loja repete a requisição por timeout de rede.	0 cobranças duplicadas
CQ-05	Confiabilidade	A loja fica fora do ar por uma hora durante as notificações.	0 eventos perdidos; mensagens na DLQ reprocessadas
CQ-06	Desempenho	Sob 100 req/s, um pagamento com cartão é criado.	$p95 \leq 2$ s
CQ-07	Desempenho	Um pagamento é confirmado; a loja é notificada.	Webhook entregue em ≤ 30 s
CQ-08	Escalabilidade	A carga sobe de 50 para 300 req/s em 5 minutos.	KEDA estabiliza em < 2 min sem erros 5xx
CQ-09	Escalabilidade	A fila fica vazia por 5 minutos.	Worker com 0 réplicas
CQ-10	Manutenibilidade	O gateway externo é trocado por outro fornecedor.	Apenas o adaptador GatewayClient é alterado
CQ-11	Manutenibilidade	O núcleo da API é alterado.	Cobertura de testes $\geq 80\%$

11 RISCOS E DÍVIDAS TÉCNICAS

Quadro 16 – Riscos técnicos

ID	Risco	Prob.	Impacto	Mitigação
R-01	Indisponibilidade ou lentidão do gateway externo	Média	Alto	Timeout, retentativa, circuit breaker e resposta 503 com Retry-After; monitorar SLA do fornecedor.
R-02	Perda de dados no Redis em falha simultânea de primário e réplica	Baixa	Alto	Persistência AOF, réplica em outra zona, snapshots diários; migração futura para banco relacional.
R-03	Lojas com webhooks instáveis encham a DLQ	Média	Médio	Alarme de DLQ, redrive manual e painel de reenvio.
R-04	Custo do cluster acima do orçamento	Média	Médio	Cluster Autoscaler, worker escalando a zero e desligamento de homologação fora do horário.
R-05	Dependência de um único fornecedor de pagamento	Baixa	Médio	Porta GatewayClient permite segundo adaptador no futuro.

Fonte: os autores (2026).

Quadro 17 – Dívidas técnicas

ID	Dívida	Plano
DT-01	Redis como único armazenamento de pagamentos.	Adotar banco relacional (PostgreSQL) quando o volume ou os requisitos de consulta exigirem.
DT-02	Sem portal do lojista; operação só pela API.	Avaliar portal em fase posterior.
DT-03	Testes de carga (CQ-06 a CQ-09) ainda não executados.	Executar com k6 no próximo sprint.
DT-04	Região única, sem recuperação de desastres.	Tratar na fase de princípios de arquitetura em nuvem.

Fonte: os autores (2026).

12 GLOSSÁRIO

Quadro 18 – Glossário

Termo	Definição
API Key	Chave secreta que identifica e autentica o lojista em cada requisição.
Circuit breaker	Padrão que interrompe temporariamente chamadas a um serviço que está falhando.
DLQ (Dead Letter Queue)	Fila que recebe mensagens não processadas após o número máximo de tentativas.
Gateway de pagamento externo	Provedor que tokeniza cartões e autoriza pagamentos com cartão e PIX junto ao sistema financeiro.
Idempotência	Propriedade pela qual repetir uma operação produz o mesmo resultado de executá-la uma vez.

Termo	Definição
KEDA	Kubernetes Event-driven Autoscaling; escala cargas de trabalho com base em métricas externas, como o tamanho de uma fila.
Load balancer	Componente que distribui as requisições entre várias instâncias da aplicação.
PAN	Primary Account Number; número do cartão de pagamento.
PIX	Meio de pagamento instantâneo brasileiro operado pelo Banco Central.
Redrive	Operação da SQS que devolve mensagens da DLQ à fila de origem.
SQS	Amazon Simple Queue Service; serviço de filas gerenciado.
Token de cartão	Identificador opaco que substitui o número do cartão, gerado pelo gateway externo.
Visibility timeout	Tempo em que uma mensagem recebida da SQS fica invisível a outros consumidores.
Webhook	Chamada HTTP feita a uma URL do destinatário quando ocorre um evento.